Ads V2

One table that tells you what every ad actually returned. Spend, DMs, booked calls, calls taken, sales, cash — for any date window.

What this is for

You run Meta ads that tell people to send a keyword in the DMs. A setter books them. A closer sells them. Ads Manager can tell you what you spent. It cannot tell you what came back.

This connects the two, keyword by keyword, so you can see which ads produce buyers rather than which ads produce clicks.

The one thing it will not do

It will not guess. If a sale cannot be connected to an ad, it shows as **unattributed** rather than being assigned to the most likely candidate. Unattributed revenue you can see is a problem you can fix. Revenue quietly credited to the wrong ad is a decision you will get wrong and never know it.

Time and difficulty

About 45 minutes, most of it waiting for accounts to create. You do not need to be technical. Where a step uses the terminal, the exact command is written out and you can paste it.

> **The fast way.** Open the project folder in Claude Code and say **“install this”**. It will do everything here except the parts that need you in a browser — creating accounts, clicking consent screens, generating tokens. It asks you for those, all at once, and handles the rest. This document is what to read while it asks, and what to come back to when something looks wrong.

Collect these first

All six. Get them before you start and the rest takes twenty minutes.

	What	Where it comes from
1	Supabase project keys	supabase.com
2	Google sign-in keys	console.cloud.google.com
3	Meta ad account id and token, one set per creator	business.facebook.com

	What	Where it comes from
4	Your ManyChat account	manychat.com
5	Your booking CRM	GoHighLevel, Calendly, whatever you use
6	Your sales tracker sheet	Google Sheets — optional, but no ROAS without it

1 The database

1. Go to **supabase.com**, sign in, and click **New project**.
2. Name it anything. Pick the region closest to you. Save the database password somewhere — you are not shown it again.
3. Wait about two minutes for it to finish setting up.
4. Open **Project Settings** → **API** and copy three things: the **Project URL**, the **anon public** key, and the **service_role** key.

> **Careful with the service_role key.** It can do anything to your database. It belongs in server settings only — never in a browser, a screenshot, or a message.

The free tier is fine to start with.

2 Sign-in

1. Go to **console.cloud.google.com**. Create a project, or use one you already have.
2. Open **APIs & Services** → **OAuth consent screen**. Choose **External**. Fill in the app name and your email. Save.
3. Open **APIs & Services** → **Credentials** → **Create credentials** → **OAuth client ID**. Application type: **Web application**.
4. Under **Authorised redirect URIs**, add both of the addresses below, exactly.
5. Copy the **Client ID** and the **Client secret**.

```
http://localhost:3000/api/auth/callback/google
https://YOUR-APP.vercel.app/api/auth/callback/google
```

> **This is the most common thing that goes wrong.** A trailing slash, a missing `/callback/google`, or `http` where it should be `https` all produce the same unhelpful “redirect_uri_mismatch” error. Check it character by character. Come back and add the second address once you know your real one (step 8).

3 Meta ad access, per creator

You need an ad account id and an access token for each person whose ads you want in the dashboard.

The **ad account id** is in Ads Manager, top left, next to the account name. It looks like `act_123456789`.

The token

Use a **System User** token. A normal token expires after about 60 days and takes your dashboard down with no warning when it does. A System User token does not expire.

1. **business.facebook.com** → **Business Settings** → **Users** → **System Users**
2. **Add**, give it a name, set the role to **Admin**.
3. **Add Assets** → **Ad Accounts**, select the ad account, and enable **Manage campaigns** (full control).
4. **Generate New Token**. Select your app. Tick **ads_read** and **ads_management**. Set the expiry to **Never**.
5. Copy it. It is shown once.

Repeat for each creator.

4 Install and configure

In the terminal, inside the project folder:

```
npm install
npm run setup
```

That creates `.env.local` and generates your security secrets. Open that file and fill in what you collected:

```
NEXT_PUBLIC_SUPABASE_URL=          <-- step 1, Project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY=     <-- step 1, anon public
SUPABASE_SERVICE_ROLE_KEY=         <-- step 1, service_role

AUTH_GOOGLE_ID=                    <-- step 2, Client ID
AUTH_GOOGLE_SECRET=                <-- step 2, Client secret
ALLOWED_EMAILS=you@example.com     <-- your Google account

META_AD_ACCOUNT_ALEX=act_123456789    <-- step 3
META_ACCESS_TOKEN_ALEX=               <-- step 3
```

`AUTH_SECRET`, `CRON_SECRET` and `WEBHOOK_SECRET` were generated for you. Leave them alone.

Describe your creators

Open `adsv2.config.json` and replace the `example` creator with a real one:

```
{
  "key": "alex",
  "name": "Alex Rivera",
  "active": true,
  "timezone": "America/Los_Angeles",
  "currency": "USD",
  "adAccountEnv": ["META_AD_ACCOUNT_ALEX"],
  "tokenEnv": ["META_ACCESS_TOKEN_ALEX"],
  "salesCalendarIds": [],
  "matchTokens": ["alex rivera", "rivera"]
}
```

Two things to get right:

- **timezone** is the ad account's *reporting* timezone in Meta — not where the person lives. A Sydney-based coach whose ad account reports on Sydney time gets `Australia/Sydney`, and the app re-cuts their days onto yours so a week-over-week comparison still means something.
- **currency** is what Meta *bills that ad account* in. It applies to ad spend only. Your sales figures are never touched by it.

Leave `salesCalendarIds` empty for now. You fill it in at step 6.

Build the database

```
npm run migrate
```

If it prints instructions instead of running, open Supabase → **SQL Editor**, paste in `supabase/01_tables.sql` and press Run, then do the same with `supabase/02_functions.sql`.

Check where you are

```
npm run doctor
```

> **Remember this one.** `npm run doctor` lists what is set up, what is not, and what each gap actually costs you. Run it whenever anything looks wrong. It is read-only, so you can run it as often as you like.

5 Connect ManyChat and your booking CRM

This is the part that turns spend into a funnel. Two webhooks. You need `WEBHOOK_SECRET` from `.env.local` for both.

ManyChat → keyword DMs

In your keyword automation, add an **External Request** action. Method POST, body JSON:

```
https://YOUR-APP/api/webhooks/manychat?secret=YOUR_WEBHOOK_SECRET

{
  "keyword":       "TRIM",
  "subscriber_id": "{{subscriber_id}}",
  "client":        "alex",
  "name":          "{{first_name}} {{last_name}}",
  "setter":        "{{assigned_admin}}"
}
```

`keyword`, `subscriber_id` and `client` are required. Without all three the DM cannot be tied to an ad, to the booking it produces, or to anyone's ad account — so the endpoint rejects it rather than recording something misleading. Rejections show up in ManyChat's own delivery log.

Booking CRM → booked calls

On your "appointment booked" automation:

```
https://YOUR-APP/api/webhooks/booking?secret=YOUR_WEBHOOK_SECRET

{
  "appointment_id":   "{{appointment.id}}",
  "calendar_id":      "{{appointment.calendar_id}}",
  "calendar_name":    "{{appointment.calendar_name}}",
  "contact_id":       "{{contact.id}}",
  "contact_name":     "{{contact.name}}",
  "start_time":       "{{appointment.start_time}}",
  "manychat_user_id": "{{contact.manychat_id}}"
}
```

> **manychat_user_id is worth more than every other optional field combined.** It is what ties the booking back to the DM, and therefore to the ad. If your CRM can carry the ManyChat subscriber id through — a hidden field on the booking form, or a parameter on the booking link — send it. Without it, bookings attribute far more weakly and many will show as unattributed.

To check either endpoint is live, open its URL in a browser. It will tell you whether your secret is right and what fields it expects.

6 Pin your sales calendars

Once a few bookings have come through:

```
npm run calendars
```

This prints the booking calendars that actually exist in your data, with how many bookings each one holds. Put the **sales** calendar ids into `salesCalendarIds` for the right creator in `adsv2.config.json`.

Only sales calls. Not onboarding calls, not coaching calls, not reschedule calendars.

> **Watch for duplicates.** CRMs make it easy to end up with “Strategy Session (AR)” and “Strategy Session - (AR)” side by side, with bookings splitting silently between them. It shows up as a booked count that is quietly too low. The script flags likely duplicates. If both are genuinely sales calls, pin both.

Until this step is done, **no booked calls are counted at all**. `npm run doctor` warns you about it.

7 The sales tracker

Optional — but this is where revenue and ROAS come from. Without it you still get spend, DMs, booked calls and show rate.

1. **console.cloud.google.com** → **APIs & Services** → **Library**, enable the **Google Sheets API**.
2. **Credentials** → **Create credentials** → **API key**. Copy it.
3. Share your sales sheet as **anyone with the link can view**.
4. The spreadsheet id is the long string in the sheet’s URL, between `/d/` and `/edit`.
5. Add `GOOGLE_SHEETS_API_KEY` and `GOOGLE_SHEETS_SPREADSHEET_ID` to `.env.local`.
6. In `adsv2.config.json`, set `salesSheet.enabled` to `true` and map your columns.

```
"columns": {
  "date":             "B",
  "prospectName":     "C",
  "manychatLink":     "D",
  "callTakenStatus":  "F",
  "outcome":          "I",
  "closer":           "J",
  "collectedRevenue": "N",
  "setter":           "P"
}
```

Just column letters, read off your own sheet. Only `date` and `prospectName` are required; anything you leave out simply produces no data for that field rather than a wrong guess.

> **The manychatLink column is the important one.** A column where setters paste the ManyChat chat link is what connects a closed sale back to the DM, and therefore to the ad. Without it, sales match on name alone, which is much weaker.

> **Two promises about your spreadsheet.** This app only ever **reads** it — it will never write to your sheet. And money is read exactly as written, never converted between currencies. Your tracker is one sheet in one currency, and “helpfully” converting it is how a $1,200 sale silently becomes $842.

8 Go live

1. Push the project folder to GitHub.
2. **vercel.com** → **Add New** → **Project**, import the repository, deploy.
3. **Project Settings** → **Environment Variables**: add every line from `.env.local`. Add `AUTH_URL`, set to your real address.
4. Go back to Google (step 2) and add the production callback address.
5. Redeploy.

The scheduled sync is already configured. It runs hourly, and a nightly check records any accuracy problems.

9 Check it actually works

Open your dashboard and confirm all four. An install that finished but shows an empty table is not finished.

	Check	If it is wrong
■	**Spend appears** for each creator	Run `npm run doctor`, then re-check the token from step 3
■	**DMs appear** after you send yourself a test keyword DM	Open the ManyChat webhook URL in a browser and check the secret
■	**A test booking appears**	`salesCalendarIds` is empty or wrong — go back to step 6
■	**npm run doctor is clean**	It will tell you what is left

When something looks wrong

What you see	What it usually is
The table is empty	Run `npm run doctor`. It will tell you.
Spend, but no DMs	The ManyChat webhook. Open its URL in a browser to test it.
DMs, but no booked calls	`salesCalendarIds` is empty or wrong. Run `npm run calendars`.
The booked count looks too low	Duplicate calendars. `npm run calendars` flags them.
Sales show as unattributed	No ManyChat link column in your sheet, so there is nothing to join on.

What you see	What it usually is
One creator vanished	Their Meta token expired. Regenerate it as a System User token (step 3).
Numbers stopped updating	`CRON_SECRET` does not match between your file and Vercel.
“redirect_uri_mismatch”	The Google callback address. Check it character by character.

Every sync writes a row to `adsv2_sync_runs`, including runs that were skipped and why. Every problem writes to `adsv2_alerts`. Look there before guessing.

The one rule to remember

Put the keyword at the end of the ad name.

```
TEST | Direct CTA | TRIM     ->   trim
Q3 Scaling (PRIMED)          ->   primed
Lead Magnet | 50 | Tempo     ->   tempo
```

That word is the whole connection between money spent and DMs received. An ad named without it still records its spend — it just can never be credited with a single DM, call, or sale.

Keep keywords unique across creators while they are live. Two people running the same word at once makes it impossible to say whose ad a DM came from.

Full documentation lives in the repository: `README.md` to get oriented, `docs/DATA-RULES.md` for what every number means and when it is allowed to change, and `CLAUDE.md` for how an AI assistant should install and maintain it.